Week 9: Sales and Customer Analytics System

The e-commerce company wants to analyze its sales data to understand customer purchasing

behavior, product performance, and revenue trends. The management team needs reports to

identify top customers, best-selling products, and profitable categories.

1. Apply SQL Aggregate Functions

Aggregate functions perform calculations on multiple records and return a single summarized

value.

I. COUNT()
II. SUM()
III. AVG()
IV. MIN()
V. MAX()

2. Generate Total Sales Reports

Tasks:

Write SQL queries to calculate:
1. Total number of orders placed.

```
SQL> SELECT COUNT(*) AS Number_of_Placed FROM ORDERS;

NUMBER_OF_PLACED
----------------
               5
```

2. Total revenue generated from sales.

```
SQL> SELECT SUM(Total_Amount) AS Total_Revenue FROM ORDERS;

TOTAL_REVENUE
-------------
         7149
```

3. Average order value.

```
SQL> SELECT ROUND(AVG(Total_Amount),2) AS Average_Order_value FROM ORDERS;

AVERAGE_ORDER_VALUE
-------------------
             1429.8
```

4. Highest and lowest order amount.

```
SQL> SELECT MAX(TOTAL_Amount) AS Highest_Order,
  2  MIN(Total_Amount) AS Lower_Orders
  3  From Orders;

HIGHEST_ORDER LOWER_ORDERS
------------- ------------
         3200          150
```

5. Total sales for a specific period

```
SQL> SELECT SUM(Total_Amount) AS Total_Revenue
  2  FROM Orders
  3  WHERE Order_Date BETWEEN DATE '2026-09-01' AND DATE '2026-09-30';

TOTAL_REVENUE
-------------
         5499
```

3. Customer Purchase Analysis

1. Total number of orders placed by each customer.

```
SQL> SELECT C.CUSTOMER_ID,C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.
CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_ORDERS DESC;

CUSTOMER_ID CUSTOMER_NAME            TOTAL_ORDERS
----------- ------------------------ ------------
        102 Priya                               1
        104 Karu                                1
        103 Kaviya                              1
        101 Ananya                              1
        105 Gobi                                1
```

2. Total amount spent by each customer.

```
SQL> SELECT C.CUSTOMER_ID,C.CUSTOMER_NAME AS CUSTOMER_NAME, NVL(SUM(O.TOTAL_AMOUNT), 0) AS TOTAL_AMOUNT_SPENT FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOM
ER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME ORDER BY TOTAL_AMOUNT_SPENT DESC;

CUSTOMER_ID CUSTOMER_NAME           TOTAL_AMOUNT_SPENT
----------- ------------------------ ------------------
        101 Ananya                                3200
        104 Karu                                  1500
        102 Priya                                 1299
        103 Kaviya                                1000
        105 Gobi                                   150
```

NVL() function replaces NULL with a specified value. Here for the customer with nopurchase the Amount Spent will be zero

.

3.Average spending per customer.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, ROUND(NVL(AVG(O.TOTAL_AMOUNT), 0), 2) AS AVERAGE_SPENDING FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID =
O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID,C.CUSTOMER_NAME ORDER BY AVERAGE_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME             AVERAGE_SPENDING
----------- ------------------------- ----------------
        101 Ananya                                3200
        104 Karu                                  1500
        102 Priya                                 1299
        103 Kaviya                                1000
        105 Gobi                                   150
```

4.Customers with maximum purchase amount.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROU
P BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) = (SELECT MAX(TOTAL_SPENDING) FROM (SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING FROM ORDERS GR
OUP BY CUSTOMER_ID));

CUSTOMER_ID CUSTOMER_NAME             TOTAL_PURCHASE_AMOUNT
----------- ------------------------- ---------------------
        101 Ananya                                   3200
```

5.Customers with fewer purchases.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME AS CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.C
USTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) <= 1 ORDER BY NUMBER_OF_ORDERS;

CUSTOMER_ID CUSTOMER_NAME             NUMBER_OF_ORDERS
----------- ------------------------- ----------------
        102 Priya                                    1
        104 Karu                                     1
        103 Kaviya                                   1
        101 Ananya                                   1
        105 Gobi                                     1
```

SQL> SELECT C.CUSTOMER_ID, C
P BY C.CUSTOMER_ID, C.CUSTOM

4. Find Top Customers Based on Purchase Amount

1. Top 3 customers based on total spending.

```
SQL>  SELECT * FROM (SELECT C.CUSTOMER_ID,CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_I
D GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC) WHERE ROWNUM <= 3;

CUSTOMER_ID CUSTOMER_NAME             TOTAL_SPENDING
----------- ------------------------- --------------
        101 Ananya                              3200
        104 Karu                                1500
        102 Priya                               1299
```

3. Customers with maximum number of orders.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_O
RDERS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP B
Y C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) = (SELECT MAX(ORDE
R_COUNT) FROM (SELECT COUNT(*) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOMER_
ID)) ORDER BY CUSTOMER_NAME;

CUSTOMER_ID CUSTOMER_NAME             NUMBER_OF_ORDERS
----------- ------------------------- ----------------
        101 Ananya                                   1
        105 Gobi                                     1
        104 Karu                                     1
        103 Kaviya                                   1
        102 Priya                                    1
```

4. Frequent customers

.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_O
RDERS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP B
Y C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) >= 2 ORDER BY NUMB
ER_OF_ORDERS DESC;

no rows selected
```

4. High-value customers.

```
SQL> SELECT C.CUSTOMER_ID,C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_OR
DERS, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON
 C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVIN
G SUM(O.TOTAL_AMOUNT) > 200 ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID CUSTOMER_NAME            NUMBER_OF_ORDERS TOTAL_SPENDING
----------- ---------------------- ---------------- --------------
        101 Ananya                                1           3200
        104 Karu                                  1           1500
        102 Priya                                 1           1299
        103 Kaviya                                1           1000
```

5. Identify Best-Selling Products

1. Products with maximum sales quantity.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_QUANTITY
_SOLD FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID G
ROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT PRODUCT_NAME         TOTAL_QUANTITY_SOLD
------- -------------------- -------------------
13       Jasmine                             150
11      Red Rose                             100
12       White Lily                           60
14      Purple Orchid                        50
15       Rose Bouquet                        30
```

2. Products generating highest revenue.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, SUM(OD.QUANTITY) AS QUANTITY_SOLD,
 SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_DETA
ILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
 ORDER BY TOTAL_REVENUE DESC;

PRODUCT PRODUCT_NAME         QUANTITY_SOLD TOTAL_REVENUE
------- -------------------- ------------- -------------
11      Red Rose                       100        325000
14      Purple Orchid                   50        125000
12       White Lily                     60         90000
13       Jasmine                       150         45000
15       Rose Bouquet                   30         11970
```

3. Least-selling products.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, NVL(SUM(OD.QUANTITY), 0) AS QUANTI
TY_SOLD FROM PRODUCT P LEFT JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODU
CT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY QUANTITY_SOLD ASC;

PRODUCT PRODUCT_NAME         QUANTITY_SOLD

15       Rose Bouquet                   30
14      Purple Orchid                   50
```

5. Products requiring promotion.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, P.PRICE, P.STOCK_QUANTITY, NVL(SUM
(OD.QUANTITY), 0) AS QUANTITY_SOLD FROM PRODUCT P LEFT JOIN ORDER_DETAILS OD
 ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME, P.PR
ICE, P.STOCK_QUANTITY HAVING NVL(SUM(OD.QUANTITY), 0) <= 50 AND P.STOCK_QUAN
TITY > 0 ORDER BY QUANTITY_SOLD, P.PRODUCT_NAME;

PRODUCT PRODUCT_NAME                 PRICE STOCK_QUANTITY QUANTITY_SOLD
------- -------------------- ---------- -------------- -------------
     15 Rose Bouquet                350             30            30
     14 Purple Orchid               120             50            50
```

6. Perform Category-wise Sales Analysis

1. Total sales generated in each category.

```
SQL> SELECT C.CATEGORY_NAME, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES FROM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN ORDER_DETAILS O
D ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME ORDER BY TOTAL_SALES DESC;

CATEGORY_NAME                                                                                       TOTAL_SALES
--------------------------------------------------------------------------------------------------- -----------
Roses                                                                                                  325000
Orchids                                                                                                125000
Lilies                                                                                                  90000
 Jasmine                                                                                                45000
 Bouquets                                                                                               11970
```

2. Number of products sold in each category.

```
SQL> SELECT C.CATEGORY_NAME, SUM(OD.QUANTITY) AS PRODUCTS_SOLD FROM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN ORDER_DETAILS OD ON P.PR
ODUCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME ORDER BY PRODUCTS_SOLD DESC;

CATEGORY_NAME                                                                                       PRODUCTS_SOLD
--------------------------------------------------------------------------------------------------- -------------
 Jasmine                                                                                                   150
Roses                                                                                                      100
Lilies                                                                                                      60
Orchids                                                                                                     50
 Bouquets                                                                                                   30
```

3. Highest revenue-generating category.

```
SQL> SELECT C.CATEGORY_NAME, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FR
OM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN ORDER_DET
AILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_N
AME HAVING SUM(OD.QUANTITY * OD.PRICE) = (SELECT MAX(CATEGORY_REVENUE) FROM
(SELECT SUM(OD2.QUANTITY * OD2.PRICE) AS CATEGORY_REVENUE FROM CATEGORY C2 J
OIN PRODUCT P2 ON C2.CATEGORY_ID = P2.CATEGORY_ID JOIN ORDER_DETAILS OD2 ON
P2.PRODUCT_ID = OD2.PRODUCT_ID GROUP BY C2.CATEGORY_ID));

CATEGORY_NAME
                         TOTAL_REVENUE
-------------------------------------------------------------------------------
------------------------ -------------
Roses
                                325000
```

4. Average sales per category.

```
SQL> SELECT C.CATEGORY_NAME, ROUND(AVG(OD.QUANTITY * OD.PRICE), 2) AS AVERAG
E_SALES FROM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN
 ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.
CATEGORY_NAME ORDER BY AVERAGE_SALES DESC;

CATEGORY_NAME   AVERAGE_SALES
--------------- -------------
Roses                  325000
Orchids                125000
Lilies                  90000
 Jasmine                45000
 Bouquets               11970
```

7. Generate Business Analytics Reports

Report 1: Sales Performance Report

- Total sales
- Number of orders
- Average order value

Highest order value

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES, COUNT(ORDER_ID) AS NUMBER_OF_ORDERS, ROUND(AVG(TOTAL_AMOUNT), 2) AS AVERAGE_ORDER_VALUE, MAX(TOTAL_AMOUNT) AS
HIGHEST_ORDER_VALUE FROM ORDERS;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE
----------- ---------------- ------------------
HIGHEST_ORDER_VALUE
-------------------
       7149                5             1429.8
               3200
```

Report 2: Customer Analytics Report

- Customer name
- Number of orders
- Total spending
- Purchase frequency

```
SQL> SELECT C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS, NVL(SUM(O.TOTAL_AMOUNT), 0) AS TOTAL_SPENDING, COUNT(O.ORDER_ID) AS PURCHASE_FREQUENCY F
ROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME            NUMBER_OF_ORDERS
------------------------ ----------------
TOTAL_SPENDING PURCHASE_FREQUENCY
-------------- ------------------
Ananya                              1
          3200                  1

Karu                                1
          1500                  1

Priya                               1
          1299                  1

Kaviya                              1
          1000                  1

Gobi                                1
           150                  1
```

Report 3: Product Performance Report

- Product name
- Quantity sold
- Revenue generated

```
SQL> SELECT P.PRODUCT_NAME,
  2         SUM(OD.QUANTITY) AS QUANTITY_SOLD,
  3         SUM(OD.QUANTITY * OD.PRICE) AS REVENUE_GENERATED
  4  FROM PRODUCT P
  5  JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID
  6  GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME
  7  ORDER BY REVENUE_GENERATED DESC;

PRODUCT_NAME         QUANTITY_SOLD
-------------------- -------------
REVENUE_GENERATED
-----------------
Red Rose                       100
           325000

Purple Orchid                   50
           125000

 White Lily                     60
            90000

 Jasmine                       150
            45000

 Rose Bouquet                   30
            11970
```

Report 4: Category Analysis Report

- Category name
- Total products sold
- Total revenue

```
SQL> SELECT C.CATEGORY_NAME, SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM CATEGORY C JOIN PRODUCT P ON C.CATEG
ORY_ID = P.CATEGORY_ID JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME ORDER BY TOTAL_REVENUE DESC;

CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------------------- -------------
Roses
                100        325000

Orchids
                 50        125000

Lilies
                 60         90000


CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------------------- -------------
 Jasmine
                150         45000

 Bouquets
                 30         11970
```